

MAKALAH
“TUGAS 5 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Ragil Anggara Kasih Budiarto

NIM:2410651045

Prodi:Teknik Informatika

Kelas: TI – C

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

DAFTAR ISI

BAB I ANALISIS MASALAH	3
1.1 Pahami Soal	3
1.2 Identifikasi masalah sinkronisasi.....	3
1.3 Desain Solusi.....	3
BAB II IMPLEMENTASI	4
2.1 Penjelasan algoritma.....	4
2.2 Potongan Kode Penting dan Dengan Penjelasan.....	7
2.3 Mengapa Memilih Mekanisme.....	7
BAB III TESTING & OUTPUT	8
3.1 Screenshot Output	8
3.2 Test Case	9
3.3 Hasil yang didapat	9
BAB IV HASIL ANALISIS.....	10
4.1 Apakah program sesuai ekspektasi.....	10
4.2 Masalah dan Solusinya.....	10
4.3 Performa Program	10
4.4 Pelajaran Yang Didapat.....	10
BAB V KESIMPULAN	11
5.1 Rangkuman Apa Yang Dipelajari	11
5.2 Tantangan Apa Yang Dihadapi	11
REFERENSI.....	12

BAB I

ANALISIS MASALAH

1.1 Pahami Soal

Soal ini tentang simulasi booking hotel

1.2 Identifikasi masalah sinkronisasi

- 5 kamar tersedia, 10 customer (thread) mencoba booking bersamaan.
- Diperlukan mekanisme agar tidak lebih dari 5 kamar digunakan sekaligus.
- Output dari banyak thread bisa saling tumpang tindih tanpa pengamanan.

1.3 Desain Solusi

- Menggunakan Semaphore untuk membatasi jumlah kamar.
- Menggunakan Mutex untuk mengamankan proses print agar output rapi.
- Tambahkan waktu acak dan retry agar simulasi lebih realistis.

BAB II IMPLEMENTASI

2.1 Penjelasan algoritma

```
GNU nano 7.2 hotel_booking.c *
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
#include <stdarg.h>

#define TOTAL_ROOMS 5
#define TOTAL_CUSTOMERS 10
#define MAX_TRIES 3

#define BOOKING_MIN 1
#define BOOKING_MAX 3
#define STAY_TIME 5
#define RETRY_WAIT 2

// Shared data
int rooms[TOTAL_ROOMS]; // 0 = kosong, 1 = terisi
pthread_mutex_t rooms_lock; // mutex untuk rooms + menjaga print rapi
sem_t rooms_sem; // semaphore untuk mengatur jumlah kamar tersisa

// Ambil timestamp [HH:MM:SS]
void timestamp(char *buf, size_t len) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    strftime(buf, len, "[%H:%M:%S]", &tm);
}

// print aman (dengan lock agar tidak saling bercampur)
void print_safe(const char *fmt, ...) {
    pthread_mutex_lock(&rooms_lock);
    char timebuf[16];
    timestamp(timebuf, sizeof(timebuf));
    printf("%s ", timebuf);

    va_list args;
    va_start(args, fmt);
```

```
GNU nano 7.2 hotel_booking.c *
    vprintf(fmt, args);
    va_end(args);

    printf("\n");
    fflush(stdout);
    pthread_mutex_unlock(&rooms_lock);
}

// Cari kamar kosong dan tandai terisi. Panggil ini **dengan** rooms_lock dipegang.
int find_and_occupy_room_locked(int customer_id) {
    for (int i = 0; i < TOTAL_ROOMS; ++i) {
        if (rooms[i] == 0) {
            rooms[i] = 1;
            return i; // index kamar 0-based
        }
    }
    return -1; // mustahil jika semaphore konsisten
}

int available_rooms_count_locked() {
    int cnt = 0;
    for (int i = 0; i < TOTAL_ROOMS; ++i) if (rooms[i] == 0) ++cnt;
    return cnt;
}

void free_room_locked(int index) {
    if (index >= 0 && index < TOTAL_ROOMS) rooms[index] = 0;
}

void *customer_thread(void *arg) {
    int id = *((int*)arg);
    free(arg); // free alokasi dari main
    char name[32];
    snprintf(name, sizeof(name), "Customer-%d", id);

    for (int attempt = 1; attempt <= MAX_TRIES; ++attempt) {
        print_safe("%s %s mencoba booking... (percobaan %d)", name, attempt);

        // Simulasi waktu proses booking (1..3 detik)
        int booking_time = (rand() % (BOOKING_MAX - BOOKING_MIN + 1)) + BOOKING_MIN;
```

```

GNU nano 7.2 hotel_booking.c *
sleep(booking_time);

// Coba ambil semaphore non-blocking
if (sem_trywait(&rooms_sem) == 0) {
    // Ada slot - temukan nomor kamar secara aman
    pthread_mutex_lock(&rooms_lock);
    int room_index = find_and_occupy_room_locked(id);
    int remaining = available_rooms_count_locked();
    pthread_mutex_unlock(&rooms_lock);

    if (room_index >= 0) {
        print_safe("✓ %s berhasil booking! (kamar #%d) (%d kamar tersisa)", name, room_index + 1, remaining);
        // Menginap
        sleep(STAY_TIME);
        // Check-out
        pthread_mutex_lock(&rooms_lock);
        free_room_locked(room_index);
        int remaining_after = available_rooms_count_locked();
        pthread_mutex_unlock(&rooms_lock);

        // Release semaphore slot
        sem_post(&rooms_sem);

        print_safe("■ %s check-out (kamar #%d) (%d kamar tersisa)", name, room_index + 1, remaining_after);
        return NULL;
    } else {
        // Seharusnya tak terjadi - tapi untuk kesigapan:
        sem_post(&rooms_sem);
        print_safe("! %s: unexpected: no free room found even after sem acquired", name);
        return NULL;
    }
} else {
    // Gagal karena kamar penuh
    print_safe("✗ %s gagal booking (kamar penuh), menunggu %d detik lalu coba lagi...", name, RETRY_WAIT);
    sleep(RETRY_WAIT);
}

print_safe("✖ %s gagal booking setelah %d percobaan.", name, MAX_TRIES);
return NULL;

```

```

GNU nano 7.2 hotel_booking.c *
}

int main(void) {
    srand(time(NULL) ^ getpid());

    // Init rooms
    for (int i = 0; i < TOTAL_ROOMS; ++i) rooms[i] = 0;
    pthread_mutex_init(&rooms_lock, NULL);
    sem_init(&rooms_sem, 0, TOTAL_ROOMS); // nilai awal = jumlah kamar

    printf("=== SISTEM BOOKING HOTEL ===\n");
    printf("Total Kamar: %d\n", TOTAL_ROOMS);

    pthread_t threads[TOTAL_CUSTOMERS];

    for (int i = 0; i < TOTAL_CUSTOMERS; ++i) {
        int *arg = malloc(sizeof(int));
        if (!arg) {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        *arg = i + 1;
        if (pthread_create(&threads[i], NULL, customer_thread, arg) != 0) {
            perror("pthread_create");
            exit(EXIT_FAILURE);
        }
        // sedikit jeda agar output awal lebih berurutan saat demo
        usleep(50000); // 50 ms
    }

    for (int i = 0; i < TOTAL_CUSTOMERS; ++i) {
        pthread_join(threads[i], NULL);
    }

    printf("\nSemua proses booking selesai.\n");

    sem_destroy(&rooms_sem);
    pthread_mutex_destroy(&rooms_lock);
    return 0;
}

```

- Semaphore (sem_t rooms_sem)
ini digunakan sebagai pengatur sumber daya ,dalam program ini berfungsi sebagai pengatur kamar yang tersedia,ini menjamin hanya 5 pelanggan yang bisa booking kamar secara bersamaan.
- Mutex (pthread_mutex_t rooms_lock)
ini adalah pengunci data bersama/critical section ini mempunyai fungsi untuk melindungi akses ke array rooms[] dan mencegah output print tercampur antar thread.

- `timestamp()`
ini untuk membuat string waktu [HH:MM:SS] untuk log output.
- `print_safe()`
ini untuk fungsi `printf()` dengan mutex agar hasil output tidak tercampur/tumpang tindih antar thread.
- `find_and_occupy_room_locked()`
ini digunakan untuk mencari kamar kosong dan menandainya terisi ,hanya dipanggil saat `rooms_lock` dikunci
- `free_room_locked()`
ini unutuk mengosongkan kamar saat costumer check-out.
- `available_rooms_count_locked()`
ini digunakan untuk menghitung sisa kamar kosong.
- `main()`
ini berfungsi untuk menganalisi semua variable,juga membuat 10 thread customer,dan menunggu hingga semua selesai `pthread_join` lalu menutup semaphore dan mutex.
- `customer_thread()`
ini berfungsi agar setiap thread mencoba untuk booking kamar,akan menunggu 1 – 3 detik.
- `sem_trywait()`
ini untuk mengecek apakah kamar tersedia jika slot semaphore > 0 maka akan ambil satu slot,jika tidak akan mencetak gagal dan tunggu 2 detik.
- `find_and_occupy_room_locked()`
ini berfungsi untuk thread mengunci `rooms_lock`,dann mencari kamar kosong dan menandainya sebagai terisi.
- Cetak output
ini untuk menampilkan hasil booking dan jumlah kamar terisi dengan `print_safe`.
- `sleep(5)`
ini digunakan untuk menjeda thread selama 5 detik.
- `free_room_locked()`
ini berfungsi setelah 5 detik ,kamar dikosongkan dan semaphore direalese `sem_post()`.

2.2 Potongan Kode Penting dan Dengan Penjelasan

- Semaphore

```
sem_t rooms_sem; // semaphore untuk mengatur jumlah kamar tersisa
```

Semaphore ini digunakan untuk mengatur berapa banyak kamar yang dapat digunakan dalam waktu yang bersamaan. Jadi saat customer hasil booking nilainya akan berkurang, dan saat checkout nilainya akan bertambah lagi.

- Mutex

```
pthread_mutex_t rooms_lock; // mutex untuk rooms + menjaga print rapi
```

Mutex ini berfungsi mengunci bagian kode yang di akses secara bersamaan, seperti data kamar dan proses cetak agar output tidak tumpang tindih dan data tetap konsisten.

2.3 Mengapa Memilih Mekanisme

- Semaphore cocok untuk mengatur sumber daya terbatas dalam program ini yaitu kamar
- Mutex diperlukan agar output antar-thread tidak tumpang tindih.

BAB III TESTING & OUTPUT

3.1 Screenshot Output

Hasil run 1

```
agilinggara@RagilAnggara: ~  
agilinggara@RagilAnggara:~/tugas5/hotel_booking$ gcc hotel_booking.c -o hotel_booking -pthread  
agilinggara@RagilAnggara:~/tugas5/hotel_booking$ ./hotel_booking  
=== SISTEM BOOKING HOTEL ===  
Total Kamar: 5  
[15:49:24] # Customer-1 mencoba booking... (percobaan 1)  
[15:49:24] # Customer-2 mencoba booking... (percobaan 1)  
[15:49:24] # Customer-3 mencoba booking... (percobaan 1)  
[15:49:24] # Customer-4 mencoba booking... (percobaan 1)  
[15:49:24] # Customer-5 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-6 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-7 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-8 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-9 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-10 mencoba booking... (percobaan 1)  
[15:49:25] # Customer-1 berhasil booking! (kamar #1) (4 kamar tersisa)  
[15:49:25] # Customer-4 berhasil booking! (kamar #2) (3 kamar tersisa)  
[15:49:26] # Customer-8 berhasil booking! (kamar #3) (2 kamar tersisa)  
[15:49:26] # Customer-10 berhasil booking! (kamar #4) (1 kamar tersisa)  
[15:49:26] # Customer-6 berhasil booking! (kamar #5) (0 kamar tersisa)  
[15:49:27] # Customer-7 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:27] # Customer-9 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:27] # Customer-2 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:27] # Customer-3 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:27] # Customer-5 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:29] # Customer-7 mencoba booking... (percobaan 2)  
[15:49:29] # Customer-9 mencoba booking... (percobaan 2)  
[15:49:29] # Customer-2 mencoba booking... (percobaan 2)  
[15:49:31] # Customer-3 mencoba booking... (percobaan 2)  
[15:49:31] # Customer-5 mencoba booking... (percobaan 2)  
[15:49:31] # Customer-9 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...  
[15:49:32] # Customer-1 check-out (kamar #1) (1 kamar tersisa)  
[15:49:32] # Customer-2 berhasil booking! (kamar #1) (0 kamar tersisa)  
[15:49:32] # Customer-4 check-out (kamar #2) (1 kamar tersisa)  
[15:49:32] # Customer-5 berhasil booking! (kamar #2) (0 kamar tersisa)  
[15:49:32] # Customer-8 check-out (kamar #3) (1 kamar tersisa)  
[15:49:32] # Customer-10 check-out (kamar #4) (2 kamar tersisa)  
[15:49:33] # Customer-6 check-out (kamar #5) (3 kamar tersisa)  
[15:49:33] # Customer-7 berhasil booking! (kamar #3) (2 kamar tersisa)  
[15:49:33] # Customer-9 mencoba booking... (percobaan 3)  
[15:49:34] # Customer-3 berhasil booking! (kamar #4) (1 kamar tersisa)  
[15:49:34] # Customer-9 berhasil booking! (kamar #5) (0 kamar tersisa)  
[15:49:37] # Customer-2 check-out (kamar #1) (1 kamar tersisa)  
[15:49:37] # Customer-5 check-out (kamar #2) (2 kamar tersisa)  
[15:49:38] # Customer-7 check-out (kamar #3) (3 kamar tersisa)  
[15:49:39] # Customer-3 check-out (kamar #4) (4 kamar tersisa)  
[15:49:39] # Customer-9 check-out (kamar #5) (5 kamar tersisa)  
Semua proses booking selesai.  
agilinggara@RagilAnggara:~/tugas5/hotel_booking$ |
```

Hasil run 2

```

agilinggara@RagilAnggara:~/tugas5/hotel_booking$ gcc hotel_booking.c -o hotel_booking -pthread
agilinggara@RagilAnggara:~/tugas5/hotel_booking$ ./hotel_booking
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5
[15:51:13] # Customer-1 mencoba booking... (percobaan 1)
[15:51:13] # Customer-2 mencoba booking... (percobaan 1)
[15:51:14] # Customer-3 mencoba booking... (percobaan 1)
[15:51:14] # Customer-4 mencoba booking... (percobaan 1)
[15:51:14] # Customer-5 mencoba booking... (percobaan 1)
[15:51:14] # Customer-6 mencoba booking... (percobaan 1)
[15:51:14] # Customer-7 mencoba booking... (percobaan 1)
[15:51:14] # Customer-8 mencoba booking... (percobaan 1)
[15:51:14] # Customer-9 mencoba booking... (percobaan 1)
[15:51:14] # Customer-10 mencoba booking... (percobaan 1)
[15:51:14] ✓ Customer-2 berhasil booking! (Kamar #1) (4 kamar tersisa)
[15:51:15] ✓ Customer-3 berhasil booking! (Kamar #2) (3 kamar tersisa)
[15:51:15] ✓ Customer-4 berhasil booking! (Kamar #3) (2 kamar tersisa)
[15:51:15] ✓ Customer-5 berhasil booking! (Kamar #4) (1 kamar tersisa)
[15:51:15] ✓ Customer-8 berhasil booking! (Kamar #5) (0 kamar tersisa)
[15:51:16] ✗ Customer-7 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...
[15:51:16] ✗ Customer-1 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...
[15:51:17] ✗ Customer-6 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...
[15:51:17] ✗ Customer-9 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...
[15:51:17] ✗ Customer-10 gagal booking (kamar penuh), menunggu 2 detik lalu coba lagi...
[15:51:18] # Customer-7 mencoba booking... (percobaan 2)
[15:51:18] # Customer-1 mencoba booking... (percobaan 2)
[15:51:18] # Customer-6 mencoba booking... (percobaan 2)
[15:51:19] # Customer-9 mencoba booking... (percobaan 2)
[15:51:19] # Customer-10 mencoba booking... (percobaan 2)
[15:51:19] # Customer-2 check-out (Kamar #1) (1 kamar tersisa)
[15:51:20] # Customer-3 check-out (Kamar #2) (2 kamar tersisa)
[15:51:20] # Customer-4 check-out (Kamar #3) (3 kamar tersisa)
[15:51:20] # Customer-5 check-out (Kamar #4) (4 kamar tersisa)
[15:51:20] ✓ Customer-6 berhasil booking! (Kamar #1) (3 kamar tersisa)
[15:51:20] # Customer-8 check-out (Kamar #5) (4 kamar tersisa)
[15:51:20] ✓ Customer-1 berhasil booking! (Kamar #2) (3 kamar tersisa)
[15:51:21] ✓ Customer-7 berhasil booking! (Kamar #3) (2 kamar tersisa)
[15:51:22] ✓ Customer-9 berhasil booking! (Kamar #4) (1 kamar tersisa)
[15:51:22] ✓ Customer-10 berhasil booking! (Kamar #5) (0 kamar tersisa)
[15:51:25] # Customer-6 check-out (Kamar #1) (1 kamar tersisa)
[15:51:25] # Customer-1 check-out (Kamar #2) (2 kamar tersisa)
[15:51:26] # Customer-7 check-out (Kamar #3) (3 kamar tersisa)
[15:51:27] # Customer-9 check-out (Kamar #4) (4 kamar tersisa)
[15:51:27] # Customer-10 check-out (Kamar #5) (5 kamar tersisa)

Semua proses booking selesai.
agilinggara@RagilAnggara:~/tugas5/hotel_booking$ |

```

3.2 Test Case

- 10 Thread customer dijalankan secara bersamaan.
- Tiap thread memiliki waktu booking yang acak.

3.3 Hasil yang didapat

- Output berbeda karena urutan eksekusi thread tidak bisa diprediksi/hasil outputnya random.
- Akan tetapi hasilnya konsisten hanya 5kamar aktif sekaligus,dan tidak ada deadlock dan race condition.

BAB IV HASIL ANALISIS

4.1 Apakah program sesuai ekspektasi

Iya, karena output menunjukkan maksimal 5 kamar aktif dan sinkronisasi berjalan baik.

4.2 Masalah dan Solusinya

- Tanpa adanya sinkronisasi, output akan berantakan dan data kamar tidak akurat.
- Dengan semaphore dan mutex, semua thread akan aman dan output menjadi lebih rapi.

4.3 Performa Program

Performa programnya stabil, tidak terjadi race condition/deadlock.

4.4 Pelajaran Yang Didapat

Pelajaran yang didapat adalah pentingnya sinkronisasi thread untuk menjaga data dan output agar tetap konsisten.

BAB V

KESIMPULAN

5.1 Rangkuman Apa Yang Dipelajari

Program ini jadi simulasi buat sistem booking hotel. Ia pakai konsep sinkronisasi thread pakai Semaphore. Itu buat batasi jumlah kamar yang bisa dipakai bareng-bareng. Lalu ada Mutex buat lindungi data. Juga biar output tetap rapi. Lewat cara begini, program bisa jalanin sepuluh thread customer sekaligus. Nggak ada race condition. Nggak ada deadlock. Dan nggak ada kesalahan data. Jadi proses booking, menginap, sama checkout berjalan aman. Konsisten juga. Ini nunjukin pentingnya sinkronisasi di pemrograman multithreading.

5.2 Tantangan Apa Yang Dihadapi

Tantangan yang saya hadapi dalam pengerjaan program ini adalah ketika mengatur urutan thread yang tidak dapat di prediksi dann juga memastikan agar output tetap rapih.

REFERENSI

1. OSIX Threads Programming: <https://computing.llnl.gov/tutorials/pthreads/>
2. Man pages: man pthread_mutex_lock, man sem_wait
3. Buku "Operating System Concepts" - Silberschatz